

Package ‘breaktest’

June 10, 2026

Version 1.0.3

Date 2024-05-03

Title Set of unit root and cointegration tests

Author Anton Skrobotov [aut, ctb],
Alexey Tsarev [aut, ctb],
Vadim Zyamalov [aut, cre, ctb]

Maintainer Vadim Zyamalov <zyamalov@ranepa.ru>

Depends R (>= 3.5.0)

Encoding UTF-8

Imports doSNOW, foreach, stringr, Rfast

Description This package contains a set of different unit root and
cointegration tests in the presence of structural breaks in the data.

License GPL (>= 2)

URL <https://github.com/d9d6ka/breaktest>

BugReports <https://github.com/d9d6ka/breaktest/issues>

Roxygen list(markdown = TRUE)

Config/roxygen2/version 8.0.0

Contents

bootstrap	2
breaktest.KP	3
coint.conf.sets	4
coint.CSS	5
coint.GH	8
coint.PR	9
coint.VECM	10
cs.bubble.emerge	11
info.criterions	12
KP.seq.statistic	13
MDF.1br	14
MDF.mlt	15
PY.statistic	15
uroot.ADF	16
uroot.CHLT	18

uroot.HLT	19
uroot.robust	19
uroot.robust.mlt	20
uroot.SADF	20

Index	26
--------------	-----------

bootstrap	<i>Functions to receive bootstrapped p-values.</i>
-----------	--

Description

bootstrap is a generic function aimed to handle objects of classes provided in this package.

Usage

```
bootstrap(obj, ...)

## S3 method for class 'bt_adf'
bootstrap(obj, iter = 999)

## S3 method for class 'bt_kpss'
bootstrap(obj, iter = 999, boot.type = "sample")

## S3 method for class 'bt_SADF'
bootstrap(obj, iter = 999)

## S3 method for class 'bt_GSADF'
bootstrap(obj, iter = 999)

## S3 method for class 'bt_mdfCHLT'
bootstrap(obj, iter = 999, y)
```

Arguments

obj	An object of one of the following classes: • bt_adf, • bt_kpss, • bt_SADF and bt_GSADF, • bt_mdfCHLT.
iter	Number of bootstrapping iterations.
boot.type	Type of bootstrapping: • "sample": sampling from residuals with replacement, • "Cavalieri-Taylor": multiplying residuals by $N(0, 1)$-distributed variable, • "Rademacher": multiplying residuals by Rademacher-distributed variable.

Details

bootstrap is a generic function aimed to handle objects of classes provided in this package. Normally one should use special function parameters to obtain bootstrapped critical values or p-values (depends on test). If later you decide to calculate one then you call bootstrap function on the object with corresponding results.

Value

Bootstrapped critical value or p-value.

breaktest.KP	<i>Kejriwal-Perron procedure of breaks number detection</i>
--------------	---

Description

Kejriwal-Perron procedure of breaks number detection

Usage

```
breaktest.KP(y, const = FALSE, breaks = 1, criterion = "bic", trim = 0.15)
```

Arguments

y	An input series of interest.
const	Whether the break in constant is allowed.
breaks	Number of breaks.
criterion	Needed information criterion: aic, bic, hq or lwz.
trim	A trimming value for a possible break date bounds.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

Value

The estimated optimal break point.

References

Kejriwal, Mohitosh, and Pierre Perron. "A Sequential Procedure to Determine the Number of Breaks in Trend with an Integrated or Stationary Noise Component: Determination of Number of Breaks in Trend." *Journal of Time Series Analysis* 31, no. 5 (September 2010): 305–28. <https://doi.org/10.1111/j.1467-9892.2010.00666.x>.

coint.conf.sets

Confidence sets for the break date in cointegrating regressions

Description

This procedure is to construct a confidence set for the change point in cointegrating regressions.

Usage

```
coint.conf.sets(
  y,
  trend = FALSE,
  zb = NULL,
  zf = NULL,
  nF = NULL,
  nL = NULL,
  conf.level = 0.9,
  trim = 0.05,
  criterion = "bic"
)
```

Arguments

y	A time series of interest.
trend	Whether the trend is to be included.
zb	I(1) regressors with break.
zf	I(1) regressors without break.
nF, nL	Number of leads and lags of z regressors. If any is NULL then both are estimated using informational criterion.
conf.level	Confidence level to obtain appropriate critical values.
trim	The trimming parameter to find the lower and upper bounds of possible break date.
criterion	A criterion for lead and lag number estimation.

Details

The function provided is for constructing confidence sets for a break date in cointegrating regressions by inverting a test for the break location, which is obtained by maximizing the weighted average of power. It is found in Kurozumi and Skrobotov (2018) that the limiting distribution of the test depends on the number of I(1) regressors whose coefficients sustain structural change and the number of I(1) regressors whose coefficients are fixed throughout the sample. By Monte Carlo simulations, the authors of the original paper showed that compared with a confidence interval developed by using the existing method based on the limiting distribution of the break point estimator under the assumption of the shrinking shift, the confidence set proposed in the present paper has a more accurate coverage rate, while the length of the confidence set is comparable.

Value

A list of confidence sets.

References

Kurozumi, Eiji, and Anton Skrobotov. “Confidence Sets for the Break Date in Cointegrating Regressions.” *Oxford Bulletin of Economics and Statistics* 80, no. 3 (2018): 514–35. <https://doi.org/10.1111/obes.12223>.

coint.CSS

KPSS-based cointegration test with multiple known structural breaks

Description

Procedure to test the presence of cointegration in the case of multiple known structural breaks using KPSS test. This procedure is based on the procedures for one and two known breaks by Carrion-i-Silvestre and Sansó (2006, 2007).

Usage

```
coint.CSS(
  y,
  x = NULL,
  const = FALSE,
  trend = FALSE,
  break.type,
  break.point,
  break.coint = FALSE,
  weakly.exog = TRUE,
  max.lags,
  max.leads,
  criterion = "bic",
  lr.kernel = "Kurozumi",
  lr.lag = NULL,
  boot.p = FALSE,
  boot.type = "sample",
  boot.iter = 999
)
```

Arguments

y	A time series of interest.
x	A matrix of explanatory stochastic regressors. If it's NULL then the result of this function is just a KPSS test for y.
const, trend	Whether a constant or trend should be included.
break.type	A single value or vector of • c: for the break in const, • t: for the break in trend, • ct: for the break in const and trend.
break.point	Array of structural break moments.
break.coint	Whether breaks in cointegrating relation are needed to be included. See Carrion-i-Silvestre & Sansó (2006) for details.

<code>weakly.exog</code>	Boolean where we specify whether the stochastic regressors are exogenous or not • TRUE: if the regressors are weakly exogenous, • FALSE: if the regressors are not weakly exogenous (DOLS is used in this case).
<code>max.lags, max.leads</code>	Scalars defining the initial number of lags and leads for DOLS.
<code>criterion</code>	Information criterion for DOLS lags and leads selection: aic, bic, hq, or lwz,
<code>lr.kernel</code>	Kernel for calculating long-run variance • Kurozumi for the Kurozumi's proposal, using Quadratic kernel (default). • Bartlett: for Bartlett kernel, • Quadratic: for Quadratic Spectral kernel,
<code>lr.lag</code>	scalar showing the bandwidth of long run variance estimator. If negative or NULL then the bandwidth is selected as in Andrews (1991).
<code>boot.p</code>	Whether bootstrapped p-values should be returned.
<code>boot.type</code>	Method of building synthetic y: • sample — using a sample from residuals, • Cavaliere-Taylor — multiplying residuals by $N(0, 1)$, • Rademacher — multiplying residuals by ± 1.
<code>boot.iter</code>	The number of bootstrap iterations,
<code>...</code>	A dummy parameter for technical purposes. Just ignore it. If you want to get results like in Carrion-i-Silvestre & Sansó (2006) then you should • provide a <code>break.point</code> of length 1; • for the specification of A set <code>const=TRUE, trend=FALSE, break.type=c("c")</code>; • for the specification of A set <code>const=TRUE, trend=TRUE, break.type=c("c")</code>; • for the specification of B set <code>const=TRUE, trend=TRUE, break.type=c("t")</code>; • for the specification of C set <code>const=TRUE, trend=TRUE, break.type=c("ct")</code>; • for the specification of D set <code>const=TRUE, trend=FALSE, break.type=c("c")</code>; • for the specification of E set <code>const=TRUE, trend=TRUE, break.type=c("ct")</code>, and <code>break.coint=TRUE</code>. If you want to get results like in Carrion-i-Silvestre & Sansó (2007) then you should • provide a <code>break.point</code> of length 2; • for the specification of AAn set <code>const=TRUE, trend=FALSE, break.type=c("c", "c")</code>; • for the specification of AA set <code>const=TRUE, trend=TRUE, break.type=c("c", "c")</code>; • for the specification of BB set <code>const=TRUE, trend=TRUE, break.type=c("t", "t")</code>; • for the specification of CC set <code>const=TRUE, trend=TRUE, break.type=c("ct", "ct")</code>; • for the specification of AB-BA set <code>const=TRUE, trend=FALSE, break.type=c("c", "t")</code>; • for the specification of AC-CA set <code>const=TRUE, trend=TRUE, break.type=c("c", "ct")</code>; • for the specification of BC-CB set <code>const=TRUE, trend=TRUE, break.type=c("t", "ct")</code>.

Details

This function is a generalization of two methods proposed in Carrion-i-Silvestre and Sansó (2006, 2007).

In Carrion-i-Silvestre and Sansó (2006) authors propose an LM-Type statistic to test the null hypothesis of cointegration allowing for the possibility of a structural break, in both the deterministic and the cointegration vector. The test has been designed to be used as a complement to the usual non-cointegration tests in order to obtain stronger evidence of cointegration. The cases of known and unknown break date are considered.

In Carrion-i-Silvestre and Sansó (2007) authors generalize the KPSS-type test to allow for two structural breaks. Seven models have been defined depending on the way that the structural breaks affect the time series behaviour.

At the first step obtained are the residuals from the regression of LHS series y on the set of deterministic components and (if any) exogenous regressors x . Deterministic components may include

- constant term,
- trend component,
- constant with break $DU_{t,\tau} = 1[t > \tau]$,
- trends with break $DT_{t,\tau} = 1[t > \tau](t - \tau)$.

If x are weakly exogenous then the regression is estimated by OLS, if not then DOLS regression is applied. The procedure also allows for the break in the cointegrating equation, in that case products of $DU_{t,\tau}$ and x are added to the regressors list.

After the residuals are obtained KPSS test statistic is calculated using kernel. If it's not set then QS kernel is used with bandwidth selection as in Andrews (1991), and with Kurozumi (2002) proposal of bandwidth limiting.

p-value (if needed) is calculated by bootstrapping procedure.

Value

An object of class `bt_kpss` containing

- `statistic`: the value of test statistic,
- `coefficients`: OLS/DOLS estimates of the coefficients,
- `se.coefs`: standard errors of the coefficients above,
- `t.stats`: t -statistics for the coefficients above,
- `residuals`: Residuals of the model,
- `t.beta`: t -statistics for beta,
- `fitted.values`: fitted values of the estimated model,
- `endog`: final y vector used in the estimated model,
- `exog`: final x matrix used in the estimated model,
- `DOLS.lags`: The estimated number of lags and leads in DOLS,
- `break.type`, `break.point`, `break.coint`: breaks specification,
- `criteria`: values of the information criteria for the estimated model,
- `lags`, `leads`: number of lags and leads in the estimated model.

References

Carrion-i-Silvestre, Josep Lluís, and Andreu Sansó. “Testing the Null of Cointegration with Structural Breaks.” *Oxford Bulletin of Economics and Statistics* 68, no. 5 (October 2006): 623–46. <https://doi.org/10.1111/j.1468-0084.2006.00180.x>.

Carrion-i-Silvestre, Josep Lluís, and Andreu Sansó. “The KPSS Test with Two Structural Breaks.” *Spanish Economic Review* 9, no. 2 (May 16, 2007): 105–27. <https://doi.org/10.1007/s10108-006-9017-8>.

Andrews, Donald W. K. “Heteroskedasticity and Autocorrelation Consistent Covariance Matrix Estimation.” *Econometrica* 59, no. 3 (1991): 817–58. <https://doi.org/10.2307/2938229>.

Kurozumi, Eiji. “Testing for Stationarity with a Break.” *Journal of Econometrics* 108, no. 1 (May 1, 2002): 63–99. [https://doi.org/10.1016/S0304-4076\(01\)00106-3](https://doi.org/10.1016/S0304-4076(01)00106-3).

coint.GH

Gregory-Hansen test for the absense of cointegration

Description

Gregory and Hansen (1996) test for the null hypothesis of no cointegration under a possible structural break at the unknown moment of time.

Usage

```
coint.GH(
  ...,
  shift = "level",
  trim = 0.15,
  max.lag = 10,
  criterion = "aic",
  add.cvals = TRUE
)
```

Arguments

...	Variables of interest.
shift	Expected break type.
trim	The trimming parameter to calculate break moment bounds.
max.lag	The maximum number of lags for the internal ADF testing.
criterion	The criterion for lag selection.
add.cvals	Whether critical values are to be returned. This argument is needed to suppress the calculation of critical values during the precalculation of tables needed for the p-values estimating.

Details

Gregory and Hansen (1996) test for the null hypothesis of no cointegration under a possible structural break at the unknown moment of time.

The authors proposed ADF- and Z-type tests, slightly modified to allow the presence of a possible regime shift. Three type of shifts are allowed:

- a shift in the constant,
- a shift in the constant with the trend included,
- and a shift in the constant and the cointegrating vector.

Critical values are calculated via the adopted MacKinnon procedure of estimating the model for the response surface.

Value

An object of type `cointGH`. It's a list of

- `shift`: shift type,
- `Za`: MZ_α statistic and c.v.,
- `Zt`: MZ_t statistic and c.v.,
- `ADF`: ADF statistic and c.v..

References

MacKinnon, James G. "Critical Values for Cointegration Tests." Working Paper. Working Paper. Economics Department, Queen's University, January 2010. <https://ideas.repec.org/p/qed/wpaper/1227.html>.

Gregory, Allan W., and Bruce E. Hansen. "Residual-Based Tests for Cointegration in Models with Regime Shifts." *Journal of Econometrics* 70, no. 1 (January 1, 1996): 99–126. [https://doi.org/10.1016/0304-4076\(69\)41685-7](https://doi.org/10.1016/0304-4076(69)41685-7).

coint.PR

A set of residual based tests for cointegration

Description

A set of residual based tests for cointegration

Usage

```
coint.PR(y, x, deter, kmin = 0, signif = 0.05)
```

Arguments

<code>y, x</code>	Variables of interest. <code>x</code> can be a matrix of several variables.
<code>deter</code>	A value equal to • 1: quasi-demeaned <code>y</code> and <code>x</code>, • 2: quasi-detrended <code>y</code> and <code>x</code>, • 3: quasi-demeaned <code>y</code> and quasi-detrended <code>x</code>.
<code>kmin</code>	A minimum number of lags to be used in the tests.

Details

Perron and Rodríguez (2016) provide generalized least-squares (GLS) detrended versions of single-equation static regression or residuals-based tests for testing whether or not non-stationary time series are cointegrated. Their approach is to consider nearly optimal tests for unit roots and to apply them in the cointegration context. Authors derive the local asymptotic power functions of all tests considered for a triangular data-generating process, imposing a directional restriction such that the regressors are pure integrated processes. Their GLS versions of the tests do indeed provide substantial power improvements over their ordinary least-squares counterparts.

The code provided is the original GAUSS code by Perron and Rodríguez ported to R.

Value

A list of:

- 7x1-matrix of test statistics values,
- estimated number of lags.

References

Perron, Pierre, and Gabriel Rodríguez. “Residuals-based Tests for Cointegration with Generalized Least-squares Detrended Data.” *The Econometrics Journal* 19, no. 1 (February 1, 2016): 84–111. <https://doi.org/10.1111/ectj.12056>.

coint.VECM

Test of the cointegration rank with a possible break in trend

Description

This procedure is aimed on the problem of testing for the cointegration rank of a vector autoregressive process in the case where a trend break may potentially be present in the data.

Usage

```
coint.VECM(y, r, max.lag, trim = 0.15)
```

Arguments

y	A matrix of n VAR variables.
r	The cointegration rank tested against the alternative of n .
max.lag	The maximum number of lags.
trim	The trimming parameter to find the lower and upper bounds of possible break dates.

Details

This procedure is aimed on the problem of testing for the cointegration rank of a vector autoregressive process in the case where a trend break may potentially be present in the data.

The test is based on estimating the quasi log likelihood for two situations, with break, and without it. The one with the smallest value is considered to be the result.

The code provided is the original GAUSS code by Harris et al. ported to R.

Value

A list of:

- the indicator of the rejection of null.
- the estimated break point.
- the estimated lag number.

References

Harris, David, Stephen J. Leybourne, and A. M. Robert Taylor. “Tests of the Co-Integration Rank in VAR Models in the Presence of a Possible Break in Trend at an Unknown Point.” *Journal of Econometrics, Innovations in Multiple Time Series Analysis*, 192, no. 2 (June 1, 2016): 451–67. <https://doi.org/10.1016/j.jeconom.2016.02.010>.

cs.bubble.emerge	<i>Confidence set for the emergence, collapse, and restore date of a bubble</i>
------------------	---

Description

Confidence set for the emergence, collapse, and restore date of a bubble

Usage

```
cs.bubble.emerge(y, full_N, phi_a, s2, trim = 0.1)
```

```
cs.bubble.collapse(y, lambda_e, full_N, phi_a, phi_b, s2, trim = 0.1)
```

```
cs.bubble.recovery(y, full_N, lambda_e, lambda_c, phi_a, phi_b, s2, trim = 0.1)
```

```
segments.AR1(y, trim = 0.1)
```

```
segments.NBCN(y)
```

Arguments

y	Numeric vector (levels).
full_N	Full sample size.
phi_a	AR coefficient before collapse (explosive phase).
s2	Residual variance.
trim	Trimming parameter.
lambda_e	Scalar, fraction for critical value scaling.
phi_b	AR coefficient after collapse (recovery phase).
lambda_c	Scalar $\lambda_c = T_c / T_{yall}$.

Details

Both the "estimated" and "true" parameter variants are handled via the `phi_a`, `phi_b`, `s2` arguments, which are to be taken from the results of `segments.AR1` or `segments.NBCN` functions.

`segments.AR1` function fits

$$y_t = \phi_{i_1} * y_{t-1} * I(t \leq bp) + \phi_{i_2} * y_{t-1} * I(t > bp) + e_t$$

over all candidate breakpoints in $[N \times trim, N \times (1 - trim)]$.

`segments.NBCN` function uses a sequential three-step search:

1. Find T_c on the full series.
2. Find T_e on $y[1:(T_c+1)]$.
3. Find T_r on $y[(T_c+2):end]$. Then refits the 4-regime model to get `phi_a`, `phi_b`, and `s2`.

Value

Named list of binary vectors.

`segments.AR1` returns a named list: `bp` (breakpoint index), `phi`.

`segments.NBCN` returns a named list: `Te_est`, `Tc_est`, `Tr_est`, `phi_a`, `phi_b`, `s2`.

References

Kurozumi, Eiji, Anton Skrobotov, and Alexey Tsarev. 2022. "Time-Transformed Test for Bubbles under Non-Stationary Volatility". *Journal of Financial Econometrics*, advance online publication, 23. <https://doi.org/10.1093/jjfinec/nbac004>.

Kurozumi, Eiji, and Anton Skrobotov. 2026. "Confidence Sets for the Emergence, Collapse, and Recovery Dates of a Bubble". *arXiv:2511.16172*. Preprint, *arXiv*. <https://doi.org/10.48550/arXiv.2511.16172>.

info.criterions

Information criterions

Description

Information criterions

Usage

```
info.criterions(obj, criterion, alpha = NULL, y = NULL)
```

```
## S3 method for class 'bt_ols'
```

```
AIC(obj, k = 2, alpha = NULL, y = NULL)
```

```
## S3 method for class 'bt_ols'
```

```
BIC(obj, alpha = NULL, y = NULL)
```

```
## S3 method for class 'bt_ols'
```

```
HQIC(obj, alpha = NULL, y = NULL)
```

```
## S3 method for class 'bt_ols'
```

```
LWZ(obj, alpha = NULL, y = NULL)
```

Arguments

obj	An object of one of the class <code>bt_ols</code> . It should be pointed that <code>AR.reg</code> and <code>DOLS.reg</code> return object that belong to subclasses of <code>bt_ols</code> , so they also can be used.
criterion	One of the following string values (case insensitive): • "AIC", • "BIC", • "HQIC", • "LWZ".
alpha	The coefficient α of y_{t-1} in ADF model. Needed only for criterion modification purposes, see Ng and Perron (2001) for further information. If NULL then no modification is applied.
y	The vector of y_{t-1} in ADF model. Needed only for criterion modification purposes.

Details

Calculating the value of the following informational criterions:

- Akaike,
- Schwarz (Bayesian),
- Hannan-Quinn,
- Liu et al.

Value

The value of the desired information criterion.

References

Ng, Serena, and Pierre Perron. "Lag Length Selection and the Construction of Unit Root Tests with Good Size and Power." *Econometrica* 69, no. 6 (2001): 1519–54. <https://doi.org/10.1111/1468-0262.00256>.

KP.seq.statistic

Sequential statistic for breaks at unknown date.

Description

Sequential statistic for breaks at unknown date.

Usage

```
KP.seq.statistic(
  y,
  const = FALSE,
  breaks = 1,
  criterion = "aic",
  trim = 0.15,
  max.lag = trunc(12 * (length(y)/100)^(1/4))
)
```

Arguments

y	A time series of interest.
const	Allowing the break in constant.
breaks	A number of breaks.
criterion	Needed information criterion: aic, bic, hq or lwz.
trim	A trimming value for a possible break date bounds.
max.lag	The maximum possible lag in the model.

Details

This procedure is based on ideas of Perron & Yabu (2009).

The code provided is the original Ox code by Skrobotov (2018) ported to R.

Value

An estimated Wald statistic.

References

Kejriwal, Mohitosh, and Pierre Perron. "A Sequential Procedure to Determine the Number of Breaks in Trend with an Integrated or Stationary Noise Component: Determination of Number of Breaks in Trend." *Journal of Time Series Analysis* 31, no. 5 (September 2010): 305–28. <https://doi.org/10.1111/j.1467-9892.2010.00666.x>.

MDF.1br

MDF procedure for a single unknown break.

Description

MDF procedure for a single unknown break.

Usage

```
MDF.1br(y, const = FALSE, trend = FALSE, trim = 0.15, criterion = "bic")
```

Arguments

y	A time series of interest.
const	Whether the constant term should be included.
trend	Whether the trend term should be included.
trim	Trimming value for a possible break date bounds.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

Value

A list of sublists each containing

- The value of statistic: $MDF - GLS$, $MDF - OLS$,
- The asymptotic critical values. UR values are included as well.

MDF.mlt

*MDF procedure for multiple unknown breaks.***Description**

MDF procedure for multiple unknown breaks.

Usage

```
MDF.mlt(y, const = FALSE, breaks = 1, breaks.star = 1, trim = 0.15, ZA = FALSE)
```

Arguments

y	A time series of interest.
const	Whether the constant term should be included.
breaks	Number of breaks.
breaks.star	Number of breaks got from the Kejriwal-Perron procedure.
trim	Trimming value for a possible break date bounds.
ZA	Whether ZA variant should be used.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

Value

A list of sublists each containing

- The value of statistic: $MDF - GLS$, $MDF - OLS$,
- The asymptotic critical values. UR values are included as well.

PY.statistic

*Perron-Yabu (2009) statistic for break at unknown date.***Description**

Perron-Yabu (2009) statistic for break at unknown date.

Usage

```
PY.statistic(
  y,
  const = FALSE,
  trend = FALSE,
  criterion = "aic",
  trim = 0.15,
  max.lag = trunc(12 * (length(y)/100)^(1/4))
)
```

Arguments

<code>y</code>	A time series of interest.
<code>const, trend</code>	Allowing the break in constant or trend.
<code>criterion</code>	Needed information criterion: aic, bic, hq or lwz.
<code>trim</code>	A trimming parameter to determine the lower and upper bounds for a possible break point.
<code>max.lag</code>	The maximum possible lag in the model.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

Value

A list of the estimated Wald statistic as well as its c.v.

References

Perron, Pierre, and Tomoyoshi Yabu. “Testing for Shifts in Trend With an Integrated or Stationary Noise Component.” *Journal of Business & Economic Statistics* 27, no. 3 (July 2009): 369–96. <https://doi.org/10.1198/jbes.2009.07268>.

uroot.ADF

A simple implementation of ADF test

Description

A function for ADF test with the ability to select the number of lags. Lags are selected by informational criterions which can be modified as in Ng and Perron (2001) and Cavaliere et al. (2015).

Usage

```
uroot.ADF(
  y,
  const = TRUE,
  trend = FALSE,
  max.lag = 0,
  criterion = NULL,
  modified.criterion = FALSE,
  rescale.criterion = FALSE,
  recursive = FALSE,
  cc = 0,
  gamma = 0,
  trim = 0.15,
  boot.p = FALSE,
  boot.iter = 999
)
```


Arguments

<code>y</code>	A time series of interest.
<code>const, trend</code>	Whether a constant and trend are to be included.
<code>max.lag</code>	Maximum lag number.
<code>criterion</code>	A criterion used to select number of lags. If lag selection is not needed keep this NULL.
<code>modified.criterion</code>	Whether the unit-root test modification is needed.
<code>rescale.criterion</code>	Whether the rescaling informational criterion is needed. Designed to cope with heteroscedasticity in residuals.
<code>recursive</code>	Whether a recursive detrending should be applied. See Smeeke (2013) for details.
<code>cc</code>	A filtration parameter used to construct an autocorrelation coefficient.
<code>gamma</code>	Detrending type selection parameter. If 0 the OLS detrending is applied, if 1 the GLS detrending is applied, otherwise the autocorrelation coefficient is calculated as $1 + c^\gamma T^{-\gamma}$.
<code>trim</code>	A trimming parameter.
<code>boot.p</code>	Whether bootstrapped p-values should be returned.
<code>boot.iter</code>	The number of bootstrap iterations.

Details

A function for ADF test with the ability to select the number of lags. Lags are selected by informational criteria which can be modified as in Ng and Perron (2001) and Cavaliere et al. (2015).

It's an ordinary Augmented Dickey-Fuller test using OLS estimation under the hood. Due to the Frisch-Waugh-Lovell theorem we first detrend `y` and then apply the test to the detrended series.

Value

A list containing:

- `y`,
- `const`,
- `trend`,
- `residuals`,
- coefficient estimates,
- t-statistic value,
- critical value,
- Number of lags,
- indicator of stationarity.

References

- Cavaliere, Giuseppe, Peter C. B. Phillips, Stephan Smeekes, and A. M. Robert Taylor. “Lag Length Selection for Unit Root Tests in the Presence of Nonstationary Volatility.” *Econometric Reviews* 34, no. 4 (April 21, 2015): 512–36. <https://doi.org/10.1080/07474938.2013.808065>.
- Ng, Serena, and Pierre Perron. “Lag Length Selection and the Construction of Unit Root Tests with Good Size and Power.” *Econometrica* 69, no. 6 (2001): 1519–54. <https://doi.org/10.1111/1468-0262.00256>.
- Taylor, A. M. Robert. “Regression-Based Unit Root Tests With Recursive Mean Adjustment for Seasonal and Nonseasonal Time Series.” *Journal of Business & Economic Statistics* 20, no. 2 (April 2002): 269–81. <https://doi.org/10.1198/073500102317352001>.
- MacKinnon, James G. “Critical Values for Cointegration Tests.” Working Paper. Economics Department, Queen’s University, January 2010. <https://ideas.repec.org/p/qed/wpaper/1227.html>.
- Smeekes, Stephan. “Detrending Bootstrap Unit Root Tests.” *Econometric Reviews* 32, no. 8 (July 2013): 869–91. <https://doi.org/10.1080/07474938.2012.690693>.
- Elliott, Graham, Thomas J. Rothenberg, and James H. Stock. “Efficient Tests for an Autoregressive Unit Root.” *Econometrica* 64, no. 4 (1996): 813–36. <https://doi.org/10.2307/2171846>.

uroot.CHLT

MDF test for a single break and possible heteroscedasticity

Description

MDF test for a single break and possible heteroscedasticity

Usage

```
uroot.CHLT(y, max.lag = NULL, trim = 0.15, boot.cv = FALSE, boot.iter = 999)
```

Arguments

y	A time series of interest.
max.lag	The maximum possible lag.
trim	Trimming parameter for lag selection
boot.cv	Whether we need bootstrapped critical values.
boot.iter	Number of bootstrap iterations.

Details

The code provided is the original GAUSS code by Cavaliere et al. ported to R.

Value

An object of type `mdfCHLT`. It’s a list of four sublists each containing:

- The value of MZ_α , MSB , MZ_t , or ADF ,
- The asymptotic c.v.,
- The bootstrapped c.v.

References

Cavaliere, Giuseppe, David I. Harvey, Stephen J. Leybourne, and A.M. Robert Taylor. "Testing for Unit Roots in the Presence of a Possible Break in Trend and Nonstationary Volatility." *Econometric Theory* 27, no. 5 (October 2011): 957–91. <https://doi.org/10.1017/S0266466610000605>.

uroot.HLT

Unit root testing procedure under a single structural break.

Description

Unit root testing procedure under a single structural break.

Usage

```
uroot.HLT(y, const = FALSE, trim = 0.15)
```

Arguments

y	A time series of interest.
const	Whether a constant should be included.
trim	The trimming parameter to find the lower and upper bounds of possible break dates.

Value

The value of test statistic.

References

Harvey, David I., Stephen J. Leybourne, and A. M. Robert Taylor. "Unit Root Testing under a Local Break in Trend." *Journal of Econometrics* 167, no. 1 (2012): 140–67.

uroot.robust

A wrapping function around [MDF.lbr](#).

Description

A wrapping function around [MDF.lbr](#).

Usage

```
uroot.robust(
  y,
  const = FALSE,
  trend = FALSE,
  season = FALSE,
  trim = 0.15,
  criterion = "bic"
)
```

Arguments

<code>y</code>	A time series of interest.
<code>const, trend</code>	Whether the constant term and trend should be included.
<code>season</code>	Whether the seasonal adjustment is needed.
<code>trim</code>	Trimming value for a possible break date bounds.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

<code>uroot.robust.mlt</code>	A wrapping function around breaktest.KP and MDF.mlt .
-------------------------------	---

Description

A wrapping function around [breaktest.KP](#) and [MDF.mlt](#).

Usage

```
uroot.robust.mlt(y, const = FALSE, season = FALSE, breaks = 2, trim = 0.15)
```

Arguments

<code>y</code>	A series of interest.
<code>const</code>	Whether the constant term should be included.
<code>season</code>	Whether the seasonal adjustment is needed.
<code>breaks</code>	Number of breaks.
<code>trim</code>	Trimming value for a possible break date bounds.

Details

The code provided is the original Ox code by Skrobotov (2018) ported to R.

<code>uroot.SADF</code>	<i>Supremum ADF tests</i>
-------------------------	---------------------------

Description

Supremum ADF tests

Usage

```
uroot.SADF(  
  y,  
  trim = 0.01 + 1.8/sqrt(length(y)),  
  const = TRUE,  
  add.p.value = TRUE,  
  boot.p = FALSE,  
  boot.iter = 999  
)  
  
uroot.GSADF(  
  y,  
  trim = 0.01 + 1.8/sqrt(length(y)),  
  const = TRUE,  
  add.p.value = TRUE,  
  boot.p = FALSE,  
  boot.iter = 999  
)  
  
uroot.STADF(  
  y,  
  trim = 0.01 + 1.8/sqrt(length(y)),  
  const = FALSE,  
  omega.est = TRUE,  
  truncated = TRUE,  
  is.reindex = TRUE,  
  ksi.input = "auto",  
  hc = 1,  
  pc = 1,  
  add.p.value = TRUE  
)  
  
uroot.GSTADF(  
  y,  
  trim = 0.01 + 1.8/sqrt(length(y)),  
  const = FALSE,  
  omega.est = TRUE,  
  truncated = TRUE,  
  is.reindex = TRUE,  
  ksi.input = "auto",  
  hc = 1,  
  pc = 1,  
  add.p.value = TRUE  
)  
  
uroot.sb.GSADF(  
  y,  
  trim = 0.01 + 1.8/sqrt(length(y)),  
  const = TRUE,  
  alpha = 0.05,  
  iter = 999,  
  urs = TRUE,
```

```

    seed = round(10^4 * sd(y))
)

uroot.w.SADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = TRUE,
  alpha = 0.05,
  iter = 4 * 200,
  urs = TRUE,
  seed = round(10^4 * sd(y))
)

uroot.w.GSADF(
  y,
  trim = 0.01 + 1.8/sqrt(length(y)),
  const = TRUE,
  alpha = 0.05,
  iter = 4 * 200,
  urs = TRUE,
  seed = round(10^4 * sd(y))
)

```

Arguments

y	A time series of interest.
trim	The trimming parameter to find the lower and upper bounds of possible break dates.
const	Whether the constant needs to be included.
add.p.value	Whether the p-value is to be returned. This argument is needed to suppress the calculation of p-values during the precalculation of tables needed for the p-values estimating.
omega.est	Whether the variance of Nadaraya-Watson residuals should be used.
truncated	Whether the truncation of Nadaraya-Watson residuals is needed.
is.reindex	Whether the Cavaliere and Taylor (2008) time transformation is needed.
ksi.input	The value of the truncation parameter. Can be either auto or the explicit numerical value. In the former case the numeric value is estimated.
hc	The scaling parameter for Nadaraya-Watson bandwidth.
pc	The scaling parameter for the estimated truncation parameter value.
alpha	A significance level of interest.
iter	Number of iterations.
urs	Use union of rejections strategy.
seed	A seed parameter for the random number generator.

Details

This set of unit root tests are developed for testing for bubbles under time-varying non-stationary volatility. Because the limiting distribution of the seminal Phillips et al. (2011) test depends on the variance function and usually requires a bootstrap implementation under heteroskedasticity,

Kurozumi et al. (2022) construct the test based on a deformation of the time domain. The proposed test is asymptotically pivotal under the null hypothesis and its limiting distribution coincides with that of the standard test under homoskedasticity, so that the test does not require computationally extensive methods for inference.

`uroot.SADF` is a test statistic equal to the minimum value of `uroot.ADF` for subsamples starting at $t = 1$.

`GSADF.test` is a generalized version of `SADF.test`. Subsamples are allowed to start at any point between 1 and $T(1 - trim)$.

Tests with time transformation are the modified versions of the ordinary SADF and GSADF tests using Nadaraya-Watson residuals and reindexing procedure by Cavaliere and Taylor (2008).

Refactored original code by Kurozumi et al.

Value

`uroot.SADF` returns an object of type `bt_SADF`. It's a list of:

- `y`,
- `trim`,
- `const`,
- vector of t -values,
- the value of the corresponding test statistic,
- p -value if it was asked for.

`uroot.GSADF` returns an object of class `bt_SADF` and subclass `bt_GSADF`.

`uroot.STADF` returns an object of type `bt_SADF` and subclass `bt_STADF`. It's a list of:

- `y`,
- `N`: Number of observations,
- `trim`,
- `const`,
- `omega.est`,
- `truncated`,
- `is.reindex`,
- `new.index`: the vector of new indices,
- `ksi.input`,
- `hc`,
- `h.est`,
- `u.hat`,
- `pc`,
- `w.sq`,
- `t.values`: vector of t -values,
- the value of the corresponding test statistic,
- `u.hat.truncated`: truncated residuals if truncation was asked for,
- `ksi, sigma`: estimated values of the truncation parameter and resulting s.e. if `ksi.input` equals `auto`,

- `eta.hat`: the values of reindexing function if reindexing was asked for,
- `p-value` if it was asked for.

`uroot.GSTADF` returns an object of type `bt_SADF` and subclass `bt_GSTADF`.

`uroot.sb.GSADF` returns an object of class `bt_SADF` and subclass `bt_sbGSADF`. It's a list of:

- `y`,
- main parameters, such as `trim`, `const`, `alpha`, `iter`,
- `seed`, supremum SBADF-statistic values incl. bootstrapped ones,
- `p-value`,
- indicator of explosive process.

If `urs=TRUE` then some additional values included:

- `t-values`,
- GSADF bootstrapped statistics,
- bootstrapped U-values.

`uroot.w.SADF` returns an object of class `bt_SADF` and subclass `bt_wSADF`. It's a list of:

- `y`,
- `trim`,
- `const`,
- `alpha`,
- `iter`,
- `urs`,
- `seed`,
- `sigma.sq`: the estimated variance,
- `BZ.values`: a series of BZ-statistic,
- `supBZ.value`: the maximum of `supBZ.values`,
- `supBZ.bootstrap.values`: bootstrapped supremum BZ values,
- `supBZ.cr.value`: supremum BZ α critical value,
- `p.value`,
- `is.explosive`: 1 if `supBZ.value` is greater than `supBZ.cr.value`.

if `urs` is `TRUE` the following items are also included:

- vector of t -values,
- the value of the SADF test statistic,
- `SADF.bootstrap.values`: bootstrapped SADF values,
- `U.value`: union test statistic value,
- `U.bootstrap.values`: bootstrapped series of `U.value`,
- `U.cr.value`: critical value of `U.value`.

`uroot.w.GSADF` returns an object of type `bt_SADF` and subclass `bt_wGSADF`.

References

- Kurozumi, Eiji, Anton Skrobotov, and Alexey Tsarev. “Time-Transformed Test for Bubbles under Non-Stationary Volatility.” *Journal of Financial Econometrics*, April 23, 2022. <https://doi.org/10.1093/jfinec/nbac004>.
- Cavaliere, Giuseppe, and A. M. Robert Taylor. “Time-Transformed Unit Root Tests for Models with Non-Stationary Volatility.” *Journal of Time Series Analysis* 29, no. 2 (March 2008): 300–330. <https://doi.org/10.1111/j.1467-9892.2007.00557.x>.
- Harvey, David I., Stephen J. Leybourne, and Yang Zu. “Sign-Based Unit Root Tests for Explosive Financial Bubbles in the Presence of Deterministically Time-Varying Volatility.” *Econometric Theory* 36, no. 1 (February 2020): 122–69. <https://doi.org/10.1017/S0266466619000057>.

Index

AIC.bt_ols (info.criteriaions), [12](#)

BIC.bt_ols (info.criteriaions), [12](#)

bootstrap, [2](#)

breaktest.KP, [3](#), [20](#)

coint.conf.sets, [4](#)

coint.CSS, [5](#)

coint.GH, [8](#)

coint.PR, [9](#)

coint.VECM, [10](#)

cs.bubble.collapse (cs.bubble.emerge),
[11](#)

cs.bubble.emerge, [11](#)

cs.bubble.recovery (cs.bubble.emerge),
[11](#)

HQIC.bt_ols (info.criteriaions), [12](#)

info.criteriaions, [12](#)

KP.seq.statistic, [13](#)

LWZ.bt_ols (info.criteriaions), [12](#)

MDF.1br, [14](#), [19](#)

MDF.mlt, [15](#), [20](#)

PY.statistic, [15](#)

segments.AR1, [12](#)

segments.AR1 (cs.bubble.emerge), [11](#)

segments.NBCN, [12](#)

segments.NBCN (cs.bubble.emerge), [11](#)

uroot.ADF, [16](#), [23](#)

uroot.CHLT, [18](#)

uroot.GSADF, [23](#)

uroot.GSADF (uroot.SADF), [20](#)

uroot.GSTADF, [24](#)

uroot.GSTADF (uroot.SADF), [20](#)

uroot.HLT, [19](#)

uroot.robust, [19](#)

uroot.robust.mlt, [20](#)

uroot.SADF, [20](#), [23](#)

uroot.sb.GSADF, [24](#)

uroot.sb.GSADF (uroot.SADF), [20](#)

uroot.STADF, [23](#)

uroot.STADF (uroot.SADF), [20](#)

uroot.w.GSADF, [24](#)

uroot.w.GSADF (uroot.SADF), [20](#)

uroot.w.SADF, [24](#)

uroot.w.SADF (uroot.SADF), [20](#)